

Deep Learning Classification Models

Contents

Introduction	4
I Foundations	4
1 Training Workflow	4
1.1 The Architecture: Multi-Layer Perceptron (MLP)	4
1.1.1 Key Components	4
1.2 Forward Pass	4
1.2.1 Process	4
1.2.2 The Role of Forward Propagation in Training and Inference	5
1.3 Loss Function	6
1.3.1 Regression Loss Functions	6
1.3.2 Classification Loss Functions	6
1.4 Backward Propagation	6
1.5 Optimization Algorithms	7
1.5.1 Common Optimization Algorithms	7
1.6 Hyperparameters	7
1.6.1 Architectural Hyperparameters	7
1.6.2 Training Hyperparameters	8
II Architecture Reviews	8
2 DenseNet (Densely Connected Networks)	8
2.1 Executive Summary	8
3 Theoretical Evolution: From ResNet to DenseNet	9
4 Core Architectural Components	9
4.1 Dense Blocks	9
4.2 Transition Layers	10
5 Comparative Analysis with Other Architectures	11
6 Performance Variants and Parameters	11
7 Key Advantages and Limitations	12
7.1 Advantages	12
7.2 Technical Limitations	12
8 Applications in Computer Vision	12

9	MobileNet: Efficient Deep Learning for Mobile Vision	13
9.1	Executive Summary	13
10	The Genesis of MobileNet	13
11	Standard vs. Depthwise Separable Convolutions	13
11.1	Efficiency Comparison	13
12	Evolution of MobileNet Architectures	14
12.1	MobileNet V1 (2017)	14
12.2	MobileNet V2 (2018–2019)	14
12.3	MobileNet V3 (2019)	15
12.4	MobileNet V4 (2024)	15
12.5	MobileNet V5 (2025)	15
13	Architectural Breakdown (MobileNet V2 Example)	16
14	Performance Benchmarks	16
15	Real-World Applications	16
16	Conclusion	17
17	EfficientNet: Architecture, Compound Scaling, and Performance Benchmarks	17
17.1	Executive Summary	17
18	The Principle of Compound Scaling	17
18.1	Dimensional Definitions	17
18.2	The Scaling Method	18
19	Architectural Framework	18
19.1	Key Structural Components	18
20	Baseline Architecture (EfficientNet-B0)	19
21	Model Variants and Performance	19
21.1	Performance Benchmarks	19
21.2	Comparison with Previous Architectures	20
22	Real-World Applications	20
23	Critical Evaluation: Advantages and Limitations	20
23.1	Advantages	20
23.2	Limitations	21
III	Evaluation Metrics	21
24	Evaluation Metrics	21
24.1	Classification Evaluation Measures	21
24.1.1	Confusion Matrix	21
24.1.2	Accuracy	22
24.1.3	Precision	22
24.1.4	Recall (Sensitivity)	23

24.1.5	F1 Score	23
24.1.6	Specificity	23
24.1.7	Receiver Operating Characteristic (ROC) Curve	23
24.1.8	Area Under the ROC Curve (AUC)	24
24.2	Evaluation Measures for Object Detection	24
24.2.1	Intersection over Union (IoU)	25
24.2.2	Mean Average Precision (mAP)	25
24.3	Evaluation Measures for Image Segmentation	25
24.3.1	Dice Similarity Coefficient (DSC)	25
24.3.2	Intersection over Union (IoU) for Segmentation	26
24.4	Regression Evaluation Measures	26
24.4.1	Mean Absolute Error (MAE)	26
24.4.2	Mean Squared Error (MSE)	27
24.4.3	Root Mean Squared Error (RMSE)	27
24.4.4	Coefficient of Determination (R^2)	27
IV	Experimental Results	28
25	Test Sample Based on 3 Models	28
25.1	Benchmarking Model Architectures on the Kaggle Sports Balls Dataset	28
25.1.1	DenseNet	28
25.1.2	MobileNet V2	28
25.1.3	EfficientNet	28
26	Summary	29
26.1	Performance Summary: Baseline (B) vs. Direct Utilization (D)	29
27	Architecture Summary	29
27.1	1. DenseNet Architecture	29
27.2	2. MobileNet V2 Architecture	29
27.3	3. EfficientNet Architecture	30
28	Architectural Takeaways	30
29	Decision Matrix	30
	Overall Conclusion	31

Introduction

This report surveys three influential convolutional neural network architectures – DenseNet, MobileNet, and EfficientNet – and evaluates their performance on a shared benchmarking task. Part I first reviews the core training mechanics common to all neural networks, providing the theoretical grounding for the architecture-specific discussion that follows. Part II examines each of the three architectures in turn, covering their design principles, structural components, and trade-offs. Part III defines the evaluation metrics used to assess classification, detection, segmentation, and regression models. Finally, Part IV applies these concepts in a hands-on benchmark, comparing all three architectures on the Kaggle Sports Balls dataset and distilling the results into a practical decision matrix.

Part I

Foundations

1 Training Workflow

1.1 The Architecture: Multi-Layer Perceptron (MLP)

A Multi-Layer Perceptron (MLP) is a foundational type of feedforward artificial neural network. It consists of an input layer, one or more hidden layers, and an output layer. Every node in one layer connects to every node in the next, using non-linear activation functions to map complex, non-linear relationships in data.

1.1.1 Key Components

- **Input Layer:** Receives the initial raw data (features). Each node represents a specific attribute, and no computation is performed here.
- **Hidden Layers:** The computational engine of the MLP. They learn abstract representations of the input data. A network with multiple hidden layers is considered a “deep” learning model.
- **Output Layer:** Generates the final prediction or result. If there are multiple outputs, the output layer will have a corresponding number of neurons.

1.2 Forward Pass

Forward Propagation (or Forward Pass) is the process of passing raw input data through the layers of a neural network to generate a prediction or output. Data move strictly in one direction: forward from the input layer, through any hidden layers, to the final output. Each neuron in the hidden layers processes the input as follows.

1.2.1 Process

Weighted Sum (Z) Each input is multiplied by its assigned weight, and a bias term is added:

$$Z = \sum_i W_i X_i + b \tag{1}$$

where

- X_i is the input,

- W_i is the corresponding weight,
- b is the bias term.

Activation Function The weighted sum Z is passed through an activation function to introduce non-linearity, allowing the network to learn complex patterns. Common activation functions include:

- **Sigmoid:**

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

The sigmoid function is a bounded, differentiable mathematical function that maps any real-valued number into an “S”-shaped curve with an output strictly between 0 and 1.

- **ReLU (Rectified Linear Unit):**

$$f(z) = \max(0, z) \quad (3)$$

ReLU is a highly efficient activation function for artificial neural networks. It passes positive inputs directly through while setting negative inputs to zero, which avoids the vanishing gradient problem.

- **Tanh (Hyperbolic Tangent):**

$$\tanh(z) = \frac{2}{1 + e^{-2z}} - 1 \quad (4)$$

The tanh function is a mathematical function that maps any real number to a value between -1 and 1 .

Output Layer The processed output of one layer immediately becomes the input of the next layer, repeating until the final output layer is reached. The final layer generates the model’s prediction. The choice of activation function depends on the task:

- **Softmax (multi-class classification):** A mathematical function used to transform a vector of raw, unnormalized scores (logits) into a proper probability distribution. It normalizes the values so that each one falls between 0 and 1, and their sum equals exactly 1:

$$\sigma(Z_i) = \frac{e^{Z_i}}{\sum_{j=1}^K e^{Z_j}} \quad (5)$$

- **Sigmoid (binary classification).**
- **Linear (regression).**

1.2.2 The Role of Forward Propagation in Training and Inference

Forward propagation acts as the model’s “thinking” or “prediction” phase:

- **During training:** The forward-propagated prediction is evaluated against the actual answer using a loss function. The resulting error is then used in a reverse process called backward propagation (backpropagation) to adjust the network’s weights and improve future predictions.
- **During testing (inference):** When the trained model is used in the real world to predict new, unseen data, only forward propagation occurs.

1.3 Loss Function

A loss function in a neural network is a mathematical tool that measures the gap between the model's prediction and the actual ground truth. It calculates a single numerical error score, where lower is better, that optimization algorithms (like gradient descent) use to adjust network weights and biases during training.

- Measures the difference between predicted and actual values.
- Guides training by helping algorithms like gradient descent update parameters.
- Used to evaluate model performance.
- Influences how the model learns based on the type of errors.
- Different loss functions are used for different tasks.

1.3.1 Regression Loss Functions

Used for predicting continuous numerical values (e.g., house prices, temperatures).

- **Mean Squared Error (MSE):** Calculates the average of the squared differences between predicted and actual values. Because the error is squared, it heavily penalizes large errors or outliers.
- **Mean Absolute Error (MAE):** Calculates the average of the absolute differences. It is less sensitive to outliers than MSE.

1.3.2 Classification Loss Functions

Used when assigning inputs to specific categories or classes (e.g., classifying an image as a cat or dog).

- **Binary Cross-Entropy:** Used for tasks with only two possible classes. It measures the performance of a classification model whose output is a probability between 0 and 1.
- **Categorical Cross-Entropy:** Used for multi-class classification. It measures the discrepancy between two probability distributions (the predicted distribution and the true distribution of the classes).

1.4 Backward Propagation

Backpropagation (or Backward Propagation) is the fundamental algorithm used to train artificial neural networks by calculating the exact gradients needed to update the model's weights and biases. It operates by applying the chain rule of calculus to the partial derivative of a loss function with respect to every single parameter in the network, moving backward from the final output layer to the initial input layer.

- **Calculating Gradients:** Using the chain rule from calculus to compute how much every weight and bias contributes to the model's total error. By evaluating these derivatives backward from the output layer to the input layer, optimization algorithms can efficiently adjust parameters to minimize the loss.
- **Output Unit Error:** Measures how much a specific neuron in the output layer contributed to the overall prediction error.

- **Hidden Unit Error:** The derivative of the loss function with respect to the pre-activation of a hidden neuron. Because hidden layers lack direct target values, their error is computed backward from the output layer using the chain rule.
- **Weight Updates:** The continuous or discrete adjustments, using an optimization algorithm (like Gradient Descent), made to the parameters (weights and biases) of a machine learning model to reduce its error.

1.5 Optimization Algorithms

Optimization is the process of adjusting network weights and biases to minimize a loss function. By iteratively applying optimization algorithms and backpropagation to compute gradients, the model learns to make highly accurate predictions based on the training data.

1.5.1 Common Optimization Algorithms

- **Batch Gradient Descent:** An iterative optimization algorithm that calculates the error for each example within the entire training dataset and updates the model's parameters only after all examples have been evaluated. It guarantees a stable, deterministic path to the global minimum for convex loss functions, but becomes computationally expensive with large datasets, making it accurate but slow.
- **Stochastic Gradient Descent (SGD):** An iterative optimization algorithm used to minimize a model's loss function by updating parameters sequentially using only one randomly selected training sample at a time. By avoiding the costly requirement of computing gradients across the entire dataset simultaneously, SGD drastically scales down computational overhead and memory demands. It serves as a cornerstone for training large-scale deep learning models efficiently, leading to fast but noisy updates.
- **RMSProp:** Resolves the issue of vanishing learning rates in earlier methods by maintaining an exponentially decaying average of squared gradients. This allows the algorithm to dynamically scale step sizes, preventing learning rates from becoming too small and stabilizing training.
- **Adam (Adaptive Moment Estimation):** Combines concepts from both Momentum and RMSProp. It keeps an exponentially decaying average of past gradients and squared gradients, and computes adaptive learning rates for each parameter. This is achieved by storing exponentially decaying averages of both past squared gradients (a technique often used for adaptive learning rates) and past gradients themselves (a method for incorporating momentum).

1.6 Hyperparameters

Hyperparameters in neural networks are manual configuration settings, defined prior to training, that govern the model's structure and the learning process itself. They dictate how the network learns, unlike internal parameters (weights and biases) that are learned automatically from the data.

1.6.1 Architectural Hyperparameters

These define the physical structure and capacity of the neural network.

- **Number of Hidden Layers:** Controls the “depth” of the network, allowing it to learn increasingly complex, hierarchical representations.

- **Neurons (Nodes) per Layer:** Controls the “width” of the network, determining the amount of information the network can memorize and process at any given stage.
- **Activation Functions:** Mathematical functions (such as ReLU, Sigmoid, or Tanh) added to neurons to introduce non-linearity, enabling the network to learn complex, non-linear patterns.
- **Optimizer:** The algorithm (such as Adam, SGD, or RMSProp) used to minimize the model’s error by adjusting the learning process over time.

1.6.2 Training Hyperparameters

These manage the optimization and learning process.

- **Learning Rate:** A hyperparameter that governs how much a machine learning model adjusts its parameters at each step of its optimization algorithm. The learning rate can determine whether a model delivers optimal performance or fails to learn during training. Often considered the most critical hyperparameter, it dictates the size of the step the model takes when updating its weights to minimize error. A rate too high causes unstable training, while a rate too low slows down convergence.
- **Batch Size:** The number of training samples processed by the model before it updates its internal parameters. Smaller batches add variability and help prevent getting stuck in local minima, while larger batches require more memory but provide smoother gradient estimates. It balances computational efficiency and model accuracy. Typical sizes are powers of 2 (e.g., 32, 64, 128) to optimize hardware utilization.
- **Number of Epochs:** The number of times the model will cycle through the entire training dataset. Too few can result in underfitting; too many can cause overfitting.
- **Weight Decay:** A regularization technique in deep learning designed to prevent overfitting by penalizing large model weights. By continuously shrinking parameters toward zero, it prevents the model from memorizing training data and encourages it to learn smoother, simpler functions that generalize better to unseen data.
- **Dropout Rate:** A regularization technique that randomly “turns off” a fraction of neurons during training. This prevents the network from becoming overly reliant on specific pathways, improving its ability to generalize to unseen data. Ranging from 0.0 to 1.0, it defines the fraction of neurons randomly deactivated (dropped) during training, forcing the model to learn more robust, generalized patterns.

Part II

Architecture Reviews

2 DenseNet (Densely Connected Networks)

2.1 Executive Summary

Densely Connected Networks (DenseNet) represent a significant advancement in convolutional neural network (CNN) architecture. First introduced by Huang *et al.* in 2017, DenseNet is characterized by a **dense connectivity pattern** where each layer receives feature maps from all preceding layers and passes its own feature maps to all subsequent layers.

Unlike Residual Networks (ResNet), which combine features through addition, DenseNet utilizes **concatenation**. This design promotes extensive feature reuse, alleviates the vanishing gradient problem, and significantly reduces the number of parameters required for high performance. However, these advantages come at the cost of increased GPU memory usage because all intermediate feature maps must be stored.

3 Theoretical Evolution: From ResNet to DenseNet

The development of DenseNet can be viewed as a natural extension of the ResNet architecture.

- **ResNet Approach:** ResNet decomposes the mapping into

$$f(x) = x + g(x),$$

where the shortcut connection allows gradients to flow more easily during training.

- **DenseNet Approach:** DenseNet replaces addition with concatenation. Each layer receives the outputs of every preceding layer:

$$x_l = H_l([x_0, x_1, \dots, x_{l-1}]),$$

where $[\cdot]$ denotes concatenation.

- **Connectivity Density:** For a network with L layers, DenseNet contains

$$\frac{L(L+1)}{2}$$

direct connections.

4 Core Architectural Components

4.1 Dense Blocks

A dense block consists of multiple convolutional layers.

- Each layer takes the concatenated outputs of all previous layers as its input.
- Each individual unit generally follows:

$$\text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{Convolution}.$$

- The **growth rate** (k) specifies how many new feature maps each layer contributes. This critical hyperparameter defines the number of output channels produced by each convolution block. Because each layer adds k new feature maps to the total, the number of input channels grows as the block goes deeper.

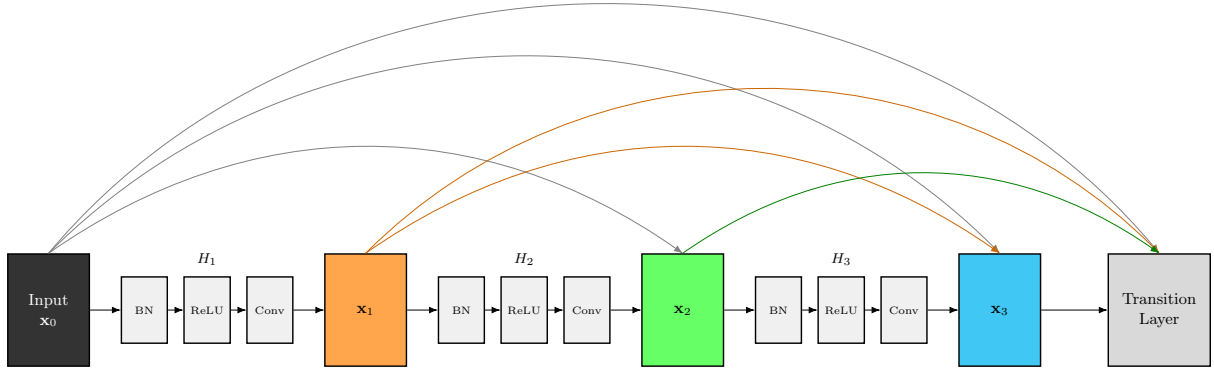


Figure 1: Dense connectivity within a DenseNet dense block. Each layer H_l applies a BN-ReLU-Conv sequence to the concatenated feature maps of all preceding layers; the block output is then passed to a transition layer for channel reduction and downsampling.

4.2 Transition Layers

Because dense blocks continuously increase the number of feature channels, **transition layers** are introduced to keep the model complexity manageable.

- **Channel Control:** A 1×1 convolution is applied to reduce the number of feature maps.
- **Downsampling:** Average pooling with a stride of 2 is used to halve the height and width of the feature maps.
- **Purpose:** Transition layers maintain computational efficiency and compactness by preventing the feature dimensionality from growing indefinitely.

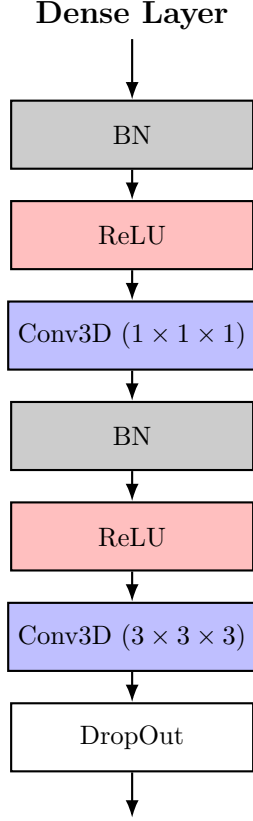


Figure 2: Dense Layer Block

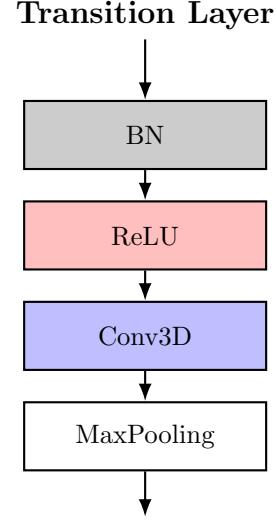


Figure 3: Transition Layer Block

Figure 4: Components of the DenseNet Architecture.

5 Comparative Analysis with Other Architectures

DenseNet offers several distinct advantages over traditional and modern CNN architectures.

Table 1: Comparison of DenseNet with Other CNN Architectures

Feature	DenseNet	ResNet	VGG	Inception
Connectivity	Dense (Concatenation)	Shortcut (Addition)	Sequential	Parallel Paths
Gradient Flow	Excellent	Good	Moderate	Good
Parameter Efficiency	High	Moderate	Low	Moderate
Feature Reuse	Extensive	Some	Minimal	Moderate
Vanishing Gradient	Mitigated	Mitigated	Prone	Mitigated
Memory Usage	High	Moderate	High	Moderate

6 Performance Variants and Parameters

DenseNet is implemented in multiple depths to accommodate different computational requirements and application domains.

Table 2: DenseNet Variants

Variant	Total Layers	Parameters	Typical Applications
DenseNet-121	121	7.98M	General classification, object detection
DenseNet-169	169	14.15M	Advanced recognition, medical imaging
DenseNet-201	201	20.01M	High-accuracy tasks, detailed extraction
DenseNet-264	264	33.34M	Complex tasks, extensive datasets

7 Key Advantages and Limitations

7.1 Advantages

- **Alleviated Vanishing Gradient:** Dense connections allow gradients to flow directly from the loss function to earlier layers, enabling the successful training of very deep neural networks.
- **Improved Feature Propagation:** Every layer has direct access to the original input signal as well as the gradients propagated from the loss function.
- **Parameter Efficiency:** By reusing previously learned feature maps instead of repeatedly learning redundant representations, DenseNet typically requires fewer parameters than ResNet while achieving comparable accuracy.
- **State-of-the-Art Performance:** DenseNet has demonstrated outstanding performance on major benchmark datasets such as ImageNet and CIFAR.

7.2 Technical Limitations

- **High Memory Consumption:** Concatenating and storing feature maps from all preceding layers requires substantial GPU memory, often necessitating memory-efficient implementations.
- **Computational and Implementation Complexity:** The increasing number of feature maps complicates both hyperparameter tuning and regularization.
- **Longer Training Time:** Due to its dense connectivity and memory overhead, DenseNet generally requires longer training times than simpler CNN architectures, limiting its suitability for some real-time applications.

8 Applications in Computer Vision

DenseNet’s ability to learn rich and reusable feature representations makes it suitable for a wide range of computer vision tasks.

- **Image Classification:** Highly effective for datasets requiring fine-grained feature discrimination.
- **Object Detection:** Serves as a powerful backbone network, producing detailed feature maps for object localization.
- **Semantic Segmentation:** Dense connectivity preserves fine-grained information, making DenseNet well suited for pixel-level segmentation tasks.

9 MobileNet: Efficient Deep Learning for Mobile Vision

9.1 Executive Summary

MobileNet is a family of convolutional neural network (CNN) architectures developed by Google, specifically designed to bring high-performance computer vision capabilities to resource-constrained environments such as smartphones, Internet of Things (IoT) devices, and embedded systems.

Introduced in 2017, MobileNet addresses the high computational and energy requirements of standard CNNs by utilizing **depthwise separable convolutions**, which significantly reduce the number of parameters and computational operations while maintaining competitive classification accuracy.

The MobileNet family has evolved through five major versions (as of June 2025), progressively incorporating advanced techniques such as inverted residuals, linear bottlenecks, Neural Architecture Search (NAS), and multi-query attention. Beyond its architectural efficiency, MobileNet has demonstrated strong real-world performance in medical diagnostics, industrial quality control, and agricultural monitoring. Despite its lightweight design, it consistently matches or exceeds the performance of much larger architectures such as VGG16 and GoogLeNet while satisfying the low-latency requirements of mobile and edge devices.

10 The Genesis of MobileNet

The development of MobileNet was motivated by the widespread adoption of smartphones and the growing demand for real-time image processing on devices with limited computational power, memory, and battery capacity.

Traditional CNN architectures rely on dense matrix multiplications, making them computationally expensive and memory intensive. MobileNet was specifically designed to overcome these limitations while maintaining competitive predictive performance.

11 Standard vs. Depthwise Separable Convolutions

The primary innovation of MobileNet is replacing standard convolutions with **depthwise separable convolutions**. This operation decomposes a conventional convolution into two separate stages.

- **Depthwise Convolution:** A single convolution filter is applied independently to each input channel, greatly reducing computational complexity by avoiding interactions across channels.
- **Pointwise Convolution (1×1):** A 1×1 convolution combines the outputs of the depthwise convolution to generate new feature representations and adjust the number of output channels.

11.1 Efficiency Comparison

The computational complexity of the two approaches is summarized below.

- **Standard Convolution**

$$K \times K \times C \times D \times H \times W$$

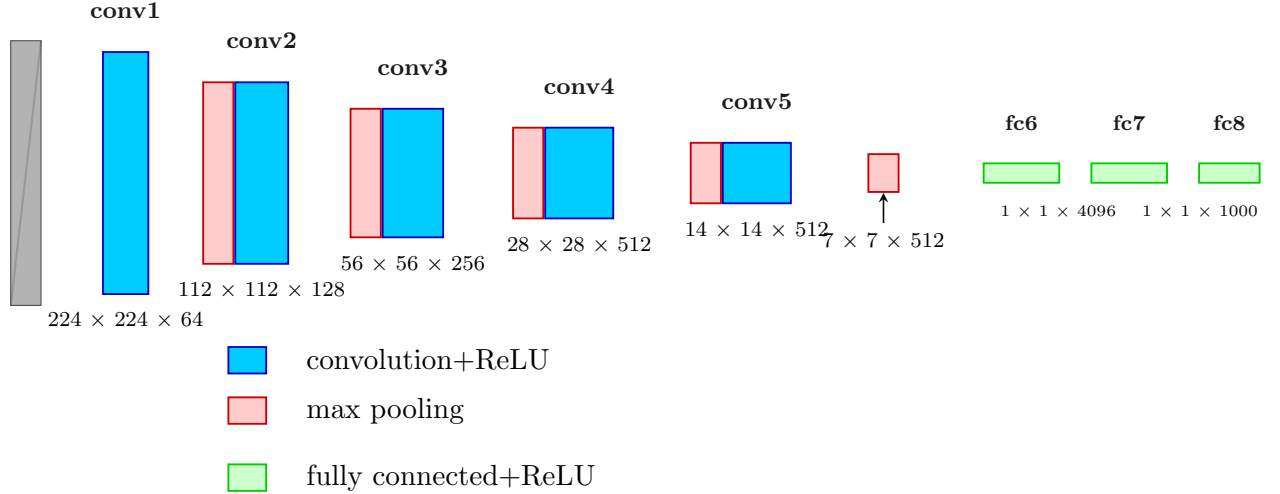
- **Depthwise Convolution**

$$K \times K \times C \times H \times W$$

where

- K = filter size,
- C = number of input channels,
- D = number of output channels,
- H = feature map height,
- W = feature map width.

By separating convolution into these two operations, MobileNet reduces the total number of parameters by nearly an order of magnitude compared with conventional CNN architectures.



12 Evolution of MobileNet Architectures

Since its introduction, Google has released five generations of MobileNet, each improving the balance between computational efficiency, model size, and predictive accuracy.

12.1 MobileNet V1 (2017)

MobileNet V1 introduced the concept of depthwise separable convolutions.

The architecture consists of 28 layers and relies heavily on optimized 1×1 convolutions, which account for approximately 95% of the computational cost and 75% of the model parameters.

Two important global hyperparameters were introduced:

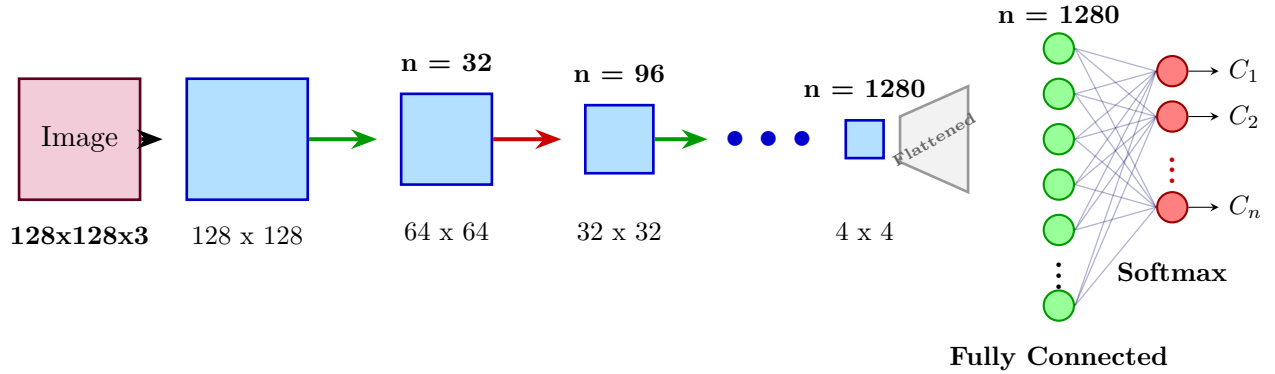
- **Width Multiplier (α):** Uniformly reduces the number of channels throughout the network, producing a thinner and more efficient model.
- **Resolution Multiplier (ρ):** Reduces the input image resolution to further decrease computational cost.

12.2 MobileNet V2 (2018–2019)

MobileNet V2 introduced two major architectural innovations:

- **Inverted Residual Blocks:** Instead of compressing inputs, these blocks expand channels before applying a depthwise convolution, then project them back down. This preserves information flow.

- **Linear Bottlenecks:** Non-linear activation functions (like ReLU) were removed from projection layers to prevent information loss in low-dimensional spaces.
- **ReLU6 Activation:** The activation output is clipped at 6, improving numerical stability for low-precision mobile hardware.



→ Preprocessing → 3x3 Conv, ReLU → Max pool 2x2

12.3 MobileNet V3 (2019)

MobileNet V3 employed **Hardware-Aware Neural Architecture Search (NAS)** to identify the most efficient architecture for mobile CPUs. Its primary innovations include:

- **Squeeze-and-Excitation (SE) Modules:** Dynamically emphasize the most informative feature maps by recalibrating channel-wise responses.
- **Hard Swish (h-swish):** A computationally efficient approximation of the Swish activation function, specifically designed for mobile processors.
- **Variants:** Released in three versions:
 - MobileNet V3-Small
 - MobileNet V3-Large
 - MobileNetEdgeTPU (optimized for Google Pixel 4)

12.4 MobileNet V4 (2024)

MobileNet V4 further expanded the architecture search space through Neural Architecture Search (NAS). Major architectural improvements include:

- **Multi-Query Attention:** Inspired by Vision Transformers (ViTs), this mechanism enhances feature representation while maintaining computational efficiency.
- **Universal Inverted Bottleneck:** A unified architectural block that generalizes previous inverted residual and bottleneck structures into a single, more flexible design.

12.5 MobileNet V5 (2025)

Released in June 2025 following the launch of Gemma 3n, V5 represents a significant scaling shift. It is reportedly 10 times larger than the largest V4 variant, though detailed technical specifications are currently pending.

13 Architectural Breakdown (MobileNet V2 Example)

The following table summarizes the typical sequence of layers in the MobileNet V2 architecture, illustrating how data flows from the input image to the final classification layer.

Table 3: MobileNet V2 Architecture

Input Size	Operator	Expansion Factor (t)	Output Channels (c)	Repeats (n)	Strides
$224 \times 224 \times 3$	Conv2D	—	32	1	2
$112 \times 112 \times 32$	Bottleneck	1	16	1	2
$112 \times 112 \times 16$	Bottleneck	6	24	2	2
$56 \times 56 \times 24$	Bottleneck	6	32	3	2
$28 \times 28 \times 32$	Bottleneck	6	64	4	2
$14 \times 14 \times 64$	Bottleneck	6	96	3	2
$14 \times 14 \times 96$	Bottleneck	6	160	3	2
$7 \times 7 \times 160$	Bottleneck	6	320	1	2
$7 \times 7 \times 320$	Conv2D (1×1)	—	1280	1	1
$7 \times 7 \times 1280$	Average Pooling	—	—	1	1
$1 \times 1 \times 1280$	Conv2D	—	k classes	1	1

14 Performance Benchmarks

MobileNet demonstrates competitive performance despite being significantly smaller than many traditional CNN architectures.

- **ImageNet:** MobileNet achieves classification accuracy comparable to VGG16 and surpasses GoogLeNet while requiring substantially fewer parameters and computational resources.
- **Task Versatility:** MobileNet has outperformed larger architectures in several challenging tasks, including fine-grained image recognition (Stanford Dogs), large-scale geolocation (PlaNet), and object detection on the COCO dataset.
- **Efficiency:** When combined with width and resolution multipliers, MobileNet consistently outperforms earlier lightweight architectures such as AlexNet and SqueezeNet.

15 Real-World Applications

The lightweight nature of MobileNet makes it an ideal choice for on-device artificial intelligence applications.

- **Healthcare:** MobileNet has been used for classifying chest X-ray images to detect COVID-19 and pneumonia with reported accuracies approaching 99%. It has also been integrated into Android applications for skin cancer detection, achieving approximately 90% accuracy.
- **Agriculture:** Mobile applications based on MobileNet assist farmers in identifying freshwater fish species (approximately 90% accuracy) and detecting ten common tomato leaf diseases using only smartphone images.
- **Industrial Safety:** MobileNet has been applied to welding defect detection from X-ray images, achieving approximately 97% accuracy while requiring fewer computational resources than ResNet-50 and VGG-19.

- **Public Health:** Trained on approximately 9,000 images, MobileNet can detect face-mask usage with an accuracy of 87.96% and determine whether masks are worn correctly with approximately 93.5% accuracy.
- **Consumer Technology:** MobileNet is widely deployed in augmented reality (AR), real-time object tracking, and facial recognition applications on smartphones and other edge devices.

16 Conclusion

MobileNet represents a paradigm shift in deep learning by emphasizing computational efficiency without sacrificing predictive performance.

Through the introduction of depthwise separable convolutions and continual architectural refinement using Neural Architecture Search (NAS), MobileNet enables sophisticated computer vision algorithms to execute directly on resource-constrained edge devices.

Its combination of low latency, compact memory footprint, and high accuracy continues to make MobileNet one of the most influential architectures for mobile, medical, agricultural, and industrial computer vision applications.

17 EfficientNet: Architecture, Compound Scaling, and Performance Benchmarks

17.1 Executive Summary

Introduced by Google AI researchers **Mingxing Tan** and **Quoc V. Le** in 2019, EfficientNet represents a fundamental shift in how convolutional neural networks (CNNs) are scaled for performance. Historically, AI models sought higher accuracy by independently increasing depth (more layers), width (more channels), or image resolution. EfficientNet replaces this haphazard approach with **compound scaling**, a principled method that scales all three dimensions simultaneously and proportionally using a fixed compound coefficient.

The resulting EfficientNet family (B0–B7) achieves state-of-the-art accuracy with significantly greater computational efficiency than earlier architectures such as ResNet, DenseNet, and GPipe. For example, EfficientNet-B7 matches GPipe’s ImageNet accuracy while being approximately 8.4 times smaller and 6.1 times faster. Owing to its excellent balance between speed and accuracy, EfficientNet has become a popular backbone for medical image analysis, object detection, and many other computer vision applications.

18 The Principle of Compound Scaling

Prior to EfficientNet, CNN scaling generally focused on increasing only one dimension (depth, width, or resolution), often leading to diminishing returns and inefficient resource utilization. EfficientNet’s primary innovation is recognizing that these three dimensions are interdependent and should be scaled together.

18.1 Dimensional Definitions

- **Width:** Refers to the number of feature channels in each layer. Increasing width enables the model to learn richer feature representations, whereas reducing width creates a lighter model.
- **Depth:** Represents the total number of network layers. Deeper models learn increasingly complex hierarchical representations but require greater computational resources.

- **Resolution:** Corresponds to the input image size. Higher-resolution images provide more detailed information but increase both memory usage and computational cost.

18.2 The Scaling Method

EfficientNet introduces a compound scaling coefficient, denoted by ϕ , to scale all three dimensions simultaneously.

The scaling strategy is defined as

$$\text{Depth} = \alpha^\phi,$$

$$\text{Width} = \beta^\phi,$$

$$\text{Resolution} = \gamma^\phi.$$

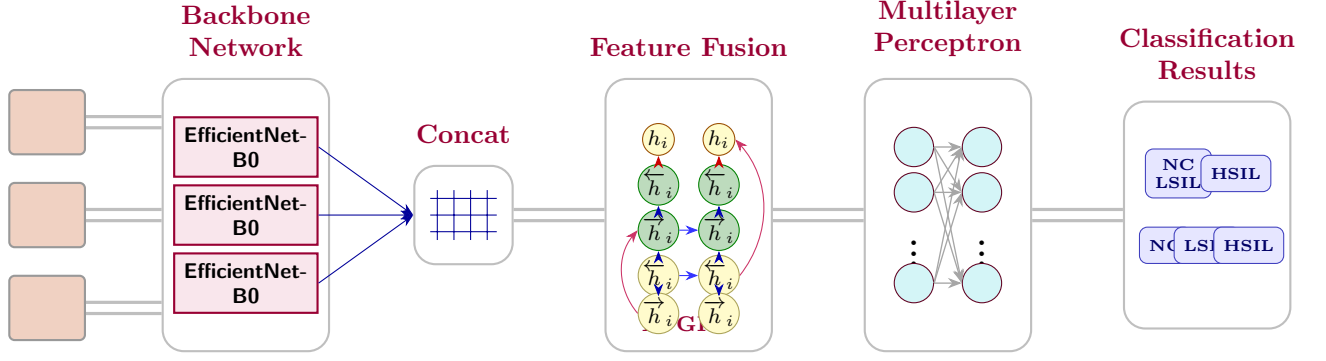
The constants α , β , and γ are determined through grid search on the baseline model to achieve the optimal balance between model accuracy and computational efficiency. This coordinated scaling strategy maximizes the utilization of available computing resources while avoiding inefficient model growth.

19 Architectural Framework

EfficientNet-B0 serves as the baseline architecture. It was discovered using AutoML and is heavily inspired by the MobileNetV2 architecture, emphasizing low parameter counts and fast inference.

19.1 Key Structural Components

- **Mobile Inverted Bottleneck (MBConv):** The fundamental building block of EfficientNet. It utilizes depthwise separable convolutions following a *narrow-wide-narrow* design.
 1. **Expansion Phase:** Expands feature maps using a 1×1 convolution.
 2. **Depthwise Convolution:** Applies either a 3×3 or 5×5 kernel independently to each channel.
 3. **Projection Phase:** Compresses the expanded feature maps back to the desired output dimension.
- **Squeeze-and-Excitation (SE) Modules:** Recalibrate channel-wise responses by emphasizing informative feature channels while suppressing less useful ones.
- **Swish Activation Function:** A smooth activation function that generally provides better optimization performance than traditional ReLU activation.
- **DropConnect:** A stochastic regularization method that randomly disables network connections during training to reduce overfitting and improve generalization.

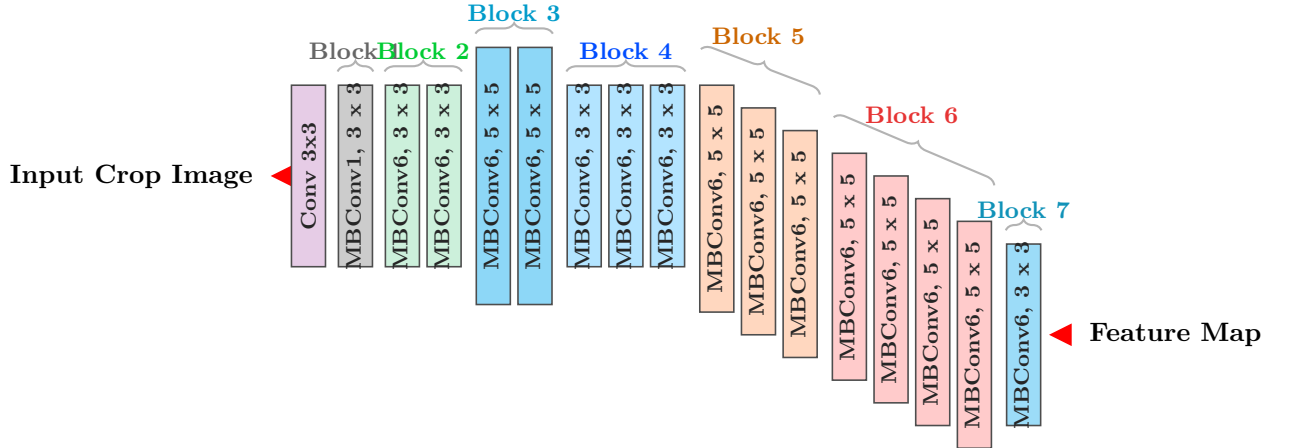


20 Baseline Architecture (EfficientNet-B0)

The following table summarizes the architecture of the EfficientNet-B0 baseline model.

Table 4: EfficientNet-B0 Architecture

Stage	Operator	Resolution	Channels	Layers
1	Conv3×3	224×224	32	1
2	MBConv1, $k = 3 \times 3$	112×112	16	1
3	MBConv6, $k = 3 \times 3$	112×112	24	2
4	MBConv6, $k = 5 \times 5$	56×56	40	2
5	MBConv6, $k = 3 \times 3$	28×28	80	3
6	MBConv6, $k = 5 \times 5$	14×14	112	3
7	MBConv6, $k = 5 \times 5$	14×14	192	4
8	MBConv6, $k = 3 \times 3$	7×7	320	1
9	Conv1×1 + Pooling + FC	7×7	1280	1



21 Model Variants and Performance

The EfficientNet family ranges from B0 to B7, with additional **EfficientNet-Lite** models designed specifically for mobile and edge-device deployment.

21.1 Performance Benchmarks

EfficientNet demonstrates excellent performance across several benchmark datasets.

- **ImageNet:** EfficientNet-B7 achieved 84.4% Top-1 accuracy and 97.3% Top-5 accuracy.

- **CIFAR-100:** Achieved 91.7% classification accuracy.
- **Flowers Dataset:** Achieved 98.8% classification accuracy.

21.2 Comparison with Previous Architectures

Table 5: EfficientNet-B7 vs. GPipe

Model	Top-1 Accuracy	Parameters	Speed
GPipe	84.3%	556 Million	Baseline
EfficientNet-B7	84.4%	66 Million	6.1× Faster

The comparison demonstrates that EfficientNet-B7 reduces the number of parameters by approximately 8.4 times while maintaining or exceeding the predictive accuracy of substantially larger architectures.

22 Real-World Applications

EfficientNet has become widely adopted across numerous scientific and industrial domains.

- **Medical Image Analysis:** EfficientNet is integrated into the MONAI (Medical Open Network for AI) framework. A specific adaptation, **Lung-EffNet**, is used to classify lung CT images for benign and malignant tumor detection with reported experimental accuracies exceeding 99%.
- **Object Detection:** EfficientNet serves as the backbone feature extractor for EfficientDet, supporting applications in autonomous driving, robotics, and drone surveillance.
- **Transfer Learning:** Due to its strong feature extraction capability, EfficientNet is commonly employed as a pretrained model, reducing both the amount of labeled data and training time required for new tasks.

23 Critical Evaluation: Advantages and Limitations

Although EfficientNet represents a significant milestone in CNN design, it also presents several practical trade-offs.

23.1 Advantages

- **High Efficiency:** Achieves state-of-the-art accuracy with substantially fewer parameters and floating-point operations (FLOPs) than ResNet and DenseNet.
- **Flexible Scaling:** The B0–B7 model family allows practitioners to select an architecture that best matches available computational resources.
- **Robust Generalization:** Techniques such as DropConnect and Squeeze-and-Excitation modules improve performance on previously unseen data.

23.2 Limitations

- **Memory Intensity:** Larger variants such as EfficientNet-B6 and EfficientNet-B7 require considerable GPU memory during training.
- **Hardware Compatibility:** MBConv layers are optimized for modern accelerators and may execute less efficiently on older CPUs and GPUs.
- **Dataset Specificity:** The compound scaling coefficients were originally optimized for ImageNet. Applying EfficientNet to substantially different datasets often requires additional fine-tuning.

Part III

Evaluation Metrics

24 Evaluation Metrics

24.1 Classification Evaluation Measures

Classification metrics are used when predicting discrete categories such as spam detection, disease diagnosis, or image classification.

24.1.1 Confusion Matrix

A Confusion Matrix summarizes prediction outcomes by comparing predicted labels with actual labels.

Table 6: General Confusion Matrix

Actual / Predicted	Positive	Negative
Positive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)

- **True Positive (TP):** Correctly predicted positive samples.
- **True Negative (TN):** Correctly predicted negative samples.
- **False Positive (FP):** Negative samples incorrectly predicted as positive.
- **False Negative (FN):** Positive samples incorrectly predicted as negative.

Almost all classification metrics are derived from the Confusion Matrix.

Example Suppose a deep learning model predicts whether an X-ray image contains pneumonia.

Table 7: Example Confusion Matrix

Actual / Predicted	Pneumonia	Healthy
Pneumonia	90	10
Healthy	15	85

Interpretation

- The model correctly identified 90 patients with pneumonia.
- It missed 10 pneumonia cases.
- It incorrectly classified 15 healthy patients as having pneumonia.
- It correctly identified 85 healthy patients.

24.1.2 Accuracy

Accuracy is a fundamental metric that evaluates the performance of a classification model. It measures the proportion of correctly predicted samples among all predictions.

Formula

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (6)$$

Advantages

- Simple to understand.
- Suitable for balanced datasets.
- Common baseline metric.

Disadvantages Accuracy can be misleading when the dataset is imbalanced. The problem arises because a model may correctly classify the majority class while failing to identify the minority class. For example, in a dataset containing 90% Class A and 10% Class B, a model predicting only Class A will still achieve 90% accuracy despite completely missing every Class B instance.

Example Using the confusion matrix above,

$$\text{Accuracy} = \frac{90 + 85}{90 + 10 + 15 + 85} = \frac{175}{200} = 0.875 \quad (7)$$

Therefore, the model correctly predicts approximately 87.5% of patients.

24.1.3 Precision

Precision measures how many predicted samples are actually positive.

Formula

$$\text{Precision} = \frac{TP}{TP + FP} \quad (8)$$

High precision indicates that false positives are low.

Advantages Precision is particularly valuable in applications such as spam filtering, fraud detection, and medical diagnosis, where false alarms are expensive.

Example

$$\text{Precision} = \frac{90}{90 + 15} = \frac{90}{105} = 0.857 \quad (9)$$

Among all patients predicted to have pneumonia, 85.7% actually have the disease.

24.1.4 Recall (Sensitivity)

Recall measures how many actual positive samples are successfully detected.

Formula

$$\text{Recall} = \frac{TP}{TP + FN} \quad (10)$$

High recall minimizes false negatives.

Advantages Recall is essential for applications such as cancer detection, disease screening, and safety-critical systems.

Example

$$\text{Recall} = \frac{90}{90 + 10} = \frac{90}{100} = 0.90 \quad (11)$$

The model detects 90% of all pneumonia patients.

24.1.5 F1 Score

The F1 Score balances precision and recall using the harmonic mean.

Formula

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (12)$$

The F1 score is preferred when datasets are imbalanced and both false positives and false negatives are important.

Example

$$\text{F1 Score} = 2 \times \frac{0.857 \times 0.90}{0.857 + 0.90} = 0.878 \quad (13)$$

The model achieves a good balance between precision and recall.

24.1.6 Specificity

Specificity measures the ability to correctly identify negative samples.

Formula

$$\text{Specificity} = \frac{TN}{TN + FP} \quad (14)$$

Advantages Specificity is important in medical testing, where correctly identifying healthy patients is essential.

Example

$$\text{Specificity} = \frac{85}{85 + 15} = 0.85 \quad (15)$$

The model correctly identifies 85% of healthy patients.

24.1.7 Receiver Operating Characteristic (ROC) Curve

The ROC curve plots the True Positive Rate (Recall) against the False Positive Rate across different classification thresholds.

Formula

$$TPR = \text{Recall} = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN} \quad (16)$$

A model with better discrimination has a ROC curve closer to the upper-left corner.

Example Suppose different probability thresholds produce the following results.

Table 8: ROC Curve Data at Different Thresholds

Threshold	Recall (TPR)	False Positive Rate (FPR)
0.9	0.45	0.02
0.7	0.70	0.08
0.5	0.90	0.15
0.3	0.98	0.30

These points form the ROC curve.

Interpretation A model whose ROC curve stays closer to the upper-left corner has better classification performance.

24.1.8 Area Under the ROC Curve (AUC)

AUC summarizes the ROC curve into a single value.

- AUC = 1.0 indicates perfect classification.
- AUC = 0.5 indicates random guessing.

Table 9: AUC Score Interpretation

AUC Range	Interpretation
0.90–1.00	Excellent
0.80–0.90	Good
0.70–0.80	Fair
0.60–0.70	Poor
0.50	Random

Interpretation

Example Suppose two image classifiers produce the following results.

Table 10: AUC Comparison Between Two Classifiers

Model	AUC
CNN-A	0.82
CNN-B	0.95

CNN-B distinguishes between positive and negative images much better than CNN-A because its AUC is closer to 1.

24.2 Evaluation Measures for Object Detection

Object detection predicts both object classes and bounding boxes.

24.2.1 Intersection over Union (IoU)

IoU measures overlap between predicted and ground-truth bounding boxes.

Formula

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}} \quad (17)$$

Typical threshold: $\text{IoU} \geq 0.5$ is considered a correct detection.

Example Suppose

- Predicted bounding box area = 120 pixels
- Ground-truth bounding box area = 100 pixels
- Overlapping area = 80 pixels

$$\text{IoU} = \frac{80}{120 + 100 - 80} = 0.571 \quad (18)$$

The prediction overlaps 57.1% with the true object. Since $\text{IoU} > 0.5$, it is typically considered a correct detection.

24.2.2 Mean Average Precision (mAP)

mAP averages precision across recall levels and object classes.

Advantages It is the standard benchmark for object detection models such as YOLO, Faster R-CNN, and SSD. Higher mAP indicates better localization and classification performance.

Example Suppose an object detection model is evaluated on three classes.

Table 11: Average Precision by Object Class

Class	Average Precision
Person	0.94
Car	0.90
Bicycle	0.86

$$\text{mAP} = \frac{0.94 + 0.90 + 0.86}{3} = 0.90 \quad (19)$$

The detector achieves an average precision of 90% across all object classes.

24.3 Evaluation Measures for Image Segmentation

Image segmentation predicts labels for every pixel.

24.3.1 Dice Similarity Coefficient (DSC)

The Dice Similarity Coefficient (DSC) is a statistical metric used to measure the overlap or similarity between two sets of data.

Formula

$$\text{DSC} = \frac{2 \times |A \cap B|}{|A| + |B|} \quad (20)$$

- Dice ranges from 0 to 1.
- Higher values indicate better segmentation quality.
- Widely used in medical image segmentation.

Example Suppose

- Ground-truth tumor mask contains 100 pixels
- Predicted tumor mask contains 110 pixels
- Overlapping pixels = 90

$$\text{DSC} = \frac{2 \times 90}{100 + 110} = 0.857 \quad (21)$$

The predicted tumor region matches the true tumor with an 85.7% overlap.

24.3.2 Intersection over Union (IoU) for Segmentation

IoU is also used for segmentation by measuring pixel overlap between predicted and true masks.

24.4 Regression Evaluation Measures

Regression metrics evaluate continuous predictions.

24.4.1 Mean Absolute Error (MAE)

MAE computes the average absolute difference between predicted and actual values.

Formula

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (22)$$

Advantages MAE is easy to interpret and less sensitive to outliers.

Example Suppose a neural network predicts house prices.

Table 12: MAE Example: Actual vs. Predicted House Prices

Actual	Predicted
300	290
250	260
400	390

Absolute errors:

$$|300 - 290| = 10, \quad |250 - 260| = 10, \quad |400 - 390| = 10$$

$$\text{MAE} = \frac{10 + 10 + 10}{3} = 10 \quad (23)$$

On average, predictions differ from the true value by 10 (based on the unit the prices are measured in).

24.4.2 Mean Squared Error (MSE)

MSE squares prediction errors before averaging.

Formula

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (24)$$

Advantages MSE strongly penalizes large errors and is differentiable, making it suitable for gradient-based optimization.

Disadvantages MSE is sensitive to outliers.

Example Suppose a neural network predicts house prices.

Table 13: MSE Example: Actual vs. Predicted House Prices

Actual	Predicted
300	290
250	270
400	380

Squared errors: 100, 400, 400.

$$\text{MSE} = \frac{100 + 400 + 400}{3} = 300 \quad (25)$$

Large errors contribute more because they are squared, making MSE sensitive to outliers.

24.4.3 Root Mean Squared Error (RMSE)

RMSE is the square root of MSE.

Formula

$$\text{RMSE} = \sqrt{\text{MSE}} \quad (26)$$

Advantages Since RMSE has the same unit as the target variable, it is easier to interpret than MSE.

Example

$$\text{RMSE} = \sqrt{300} = 17.32 \quad (27)$$

The average prediction error is approximately 17.3 units, expressed in the same units as the target variable.

24.4.4 Coefficient of Determination (R^2)

The R^2 score measures how much variance is explained by the model.

Formula

$$R^2 = 1 - \frac{SSR}{SST} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \quad (28)$$

Interpretation

- $R^2 = 1$: perfect prediction.
- $R^2 = 0$: no better than predicting the mean.
- $R^2 < 0$: worse than the baseline model.

Example Suppose

- Total Sum of Squares (SST) = 500
- Residual Sum of Squares (SSR) = 100

$$R^2 = 1 - \frac{SSR}{SST} = 1 - \frac{100}{500} = 0.80 \quad (29)$$

The model explains 80% of the variation in house prices.

Part IV

Experimental Results

25 Test Sample Based on 3 Models

25.1 Benchmarking Model Architectures on the Kaggle Sports Balls Dataset

25.1.1 DenseNet

Baseline Model

Training complete! Best Validation Accuracy: 0.8810

Direct Utilization

Training complete! Best Validation Accuracy: 0.8762

25.1.2 MobileNet V2

Baseline Model

Training complete! Best Validation Accuracy: 0.9001

Direct Utilization

Training complete! Best Validation Accuracy: 0.9033

25.1.3 EfficientNet

Baseline

Training complete! Best Validation Accuracy: 0.9147

No Baseline Model

Training complete! Best Validation Accuracy: 0.9158

26 Summary

26.1 Performance Summary: Baseline (B) vs. Direct Utilization (D)

EfficientNet achieved the highest overall performance, peaking at 91.58% accuracy using Direct Utilization.

MobileNet V2 demonstrated highly consistent results, hovering right around the 90% accuracy mark for both configurations.

DenseNet served as the lowest-performing baseline in this benchmark, maxing out at 88.10% accuracy.

Table 14: Performance Summary: Baseline (B) vs. Direct Utilization (D)

Model Architecture	Baseline Model (B)	Direct Utilization (D)	Performance Delta (Δ)
EfficientNet	91.47%	91.58%	+0.11% (D Wins)
MobileNet V2	90.01%	90.33%	+0.32% (D Wins)
DenseNet	88.10%	87.62%	-0.48% (B Wins)

Key Takeaway: Direct Utilization (D) provided a slight performance boost for both EfficientNet and MobileNet V2, whereas the Baseline configuration (B) yielded better results specifically for DenseNet.

27 Architecture Summary

27.1 1. DenseNet Architecture

DenseNet: Baseline Model (B)

Training Dynamics: Steady, smooth convergence. Validation accuracy climbs early and maintains a healthy margin above training accuracy, stabilizing near 88.10%.

Loss Behavior: Training and validation loss decrease smoothly in tandem, indicating highly stable gradient descent without signs of overfitting or erratic bounces.

DenseNet: Direct Utilization (D)

Training Dynamics: Achieves a similar plateau to the baseline, peaking at 87.62%. However, there is a visible training artifact around Epoch 5, where validation loss suddenly spikes upward while validation accuracy stalls.

Loss Behavior: The validation loss spike at Epoch 5 breaks the smooth downward trend before recovering by Epoch 6, suggesting slight optimization instability during this run.

27.2 2. MobileNet V2 Architecture

MobileNet V2: Baseline Model (B)

Training Dynamics: Reaches the 90.01% accuracy mark smoothly. The accuracy curves are incredibly tightly bound to the training metrics, showing an optimal fit.

Loss Behavior: Both loss curves scale downward uniformly. The gap between training and validation loss remains negligible throughout the 10 epochs, pointing to strong regularization.

MobileNet V2: Direct Utilization (D)

Training Dynamics: Marginally outperforms the baseline at 90.33%. However, it exhibits a distinct “bump” at Epoch 5, where validation loss sharply ticks upward and accuracy dips slightly.

Loss Behavior: The model successfully corrects the optimization anomaly by Epoch 6, but the spike reveals that Direct Utilization introduced temporary instability midway through training.

27.3 3. EfficientNet Architecture

EfficientNet: Baseline Model (B)

Training Dynamics: Rapid early learning phase, crossing the 80% validation accuracy mark by Epoch 1. It finished strong at a baseline-high of 91.47%.

Loss Behavior: Validation loss drops well below training loss initially and maintains a clean, steady convergence profile right down to its final epoch.

EfficientNet: Direct Utilization (D)

Training Dynamics: The top-performing configuration overall, maxing out at 91.58% accuracy. It shows a highly confident training trajectory with minimal gap between training and validation metrics in later epochs.

Loss Behavior: Exceptional convergence. The loss curves perfectly intersect at Epoch 10, showing full convergence right as training concludes, with zero signs of overfitting.

28 Architectural Takeaways

- **The “Epoch 5” Phenomenon:** Interestingly, both DenseNet (D) and MobileNet V2 (D) experienced an optimization disruption exactly at Epoch 5. This suggests that the Direct Utilization pipeline might benefit from a slightly tuned learning rate scheduler or warmer restarts around mid-training.
- **EfficientNet Superiority:** EfficientNet configurations completely bypassed this mid-training instability, showing robust feature extraction and smoother gradient flows on this dataset from start to finish.

29 Decision Matrix

Table 15: Decision Matrix

Model	Primary Strength	Best Hardware Fit	Ideal Data Type	Validation Result
MobileNet V2	Blazing speed, tiny size	Smartphones / Edge	Standard resolution, fast video	90.3%
DenseNet	Parameter efficiency	Standard GPUs / Limited Data	Smaller datasets, complex textures	88.1%
EfficientNet	Maximum Accuracy	Cloud / High-End GPUs	High resolution, small/varied objects	91.6% (Winner)

Overall Conclusion

Across the architectural review and the empirical benchmark, no single model dominates on every axis. DenseNet offers strong parameter efficiency and feature reuse but trails the other two architectures in raw accuracy and training stability on this dataset. MobileNet V2 remains the strongest choice when inference speed and deployment footprint are the priority, delivering consistent, near-90% accuracy at a fraction of the computational cost of larger models. EfficientNet, through compound scaling, achieved the best overall validation accuracy (91.6%) and the most stable training dynamics, making it the preferred choice when computational resources are not the binding constraint. The choice between these three architectures therefore remains a function of the deployment target – edge device, standard GPU, or cloud infrastructure – rather than a single universal winner.